

Holdpoint: Payout Autopilot Requirements

Oct 8, 2026 · Anjola Adeyemi, with controls adapted from Ringfence by GRAMMYboy

Hold only what’s at risk. Send the rest now. Amazon, eBay and Etsy built their own systems to decide which seller payouts are safe to send; Holdpoint gives every marketplace on Airwallex that system. Holdpoint checks every marketplace payout before it reaches Airwallex and answers **Clear, Reduce, Hold** or **Block**, with a reason anyone can read. Safe payouts go out on their own; the rest wait at the line, the way a plane waits at a holding point until the tower clears it. A requirements document for a build on Airwallex APIs for the Agentic Banking Hackathon.

Purpose and goals

Marketplaces collect money from buyers and pay sellers in other countries later. Payout tools send what they are told to. Three things go wrong on every run: money that can still be refunded is paid out in full, money goes to the wrong account (bad details or a fake “pay my new account” email), and a timed-out payment is retried and paid twice. The biggest marketplaces built blunt automatic holds for this (Amazon holds seller-shipped order funds until 7 days after delivery; eBay may hold payouts until delivery is confirmed or after bank details change). Most marketplaces have no team to build that, so they pay slowly or pay blind. Holdpoint makes the call before any money moves and holds only what is at risk. Primary metric: the share of payouts sent with no human review, with zero double payments and zero payouts to unverified bank changes (10 of 14 in the demo run).

Success means all of this holds:

- Every payout gets exactly one decision with a plain-language reason before anything is sent.
- Nothing moves unless deterministic code allows it. The model reads; it never sets amounts or edits bank details.
- No seller is ever paid twice, including after a timeout.
- When new information arrives (a refund, a bank-change email, a bank rejection), only the payouts it touches are re-decided.
- The run always ties out: owed = sent + kept back + held + blocked, and the internal record matches Airwallex or execution halts.

Evidence the problem is real

Figure	What it says	Source
\$118.5B	cost of failed payments in fees, labour and lost business in 2020; inaccurate beneficiary details caused about a third	LexisNexis Risk Solutions, July 2021
\$3.83T	projected 2024 GMV of the top 100 online marketplaces, up 10%	Digital Commerce 360
\$2.8B	reported US losses to business email compromise in 2024	FBI IC3, via Nacha, April 2025

Where it sits

Layer	Who	What it answers
Before the money moves	Holdpoint	Should this payout go now, in full, to this account?
While it moves	Airwallex Transfers	Can this payout be sent, and did it land?
After it lands	Leapfin (Airwallex) / the books	How is it recorded?

Airwallex's own transfer validation checks a payout can be sent; batch transfers send many at once; the Expense Policy Agent reviews card spend. None decides whether a specific seller payout should be sent now.

Domain model

- **Seller:** a marketplace seller with country, currency, trailing refund rate and one beneficiary.
- **Beneficiary:** bank details on Airwallex, validation result, last-changed time, any pending change request, any bank rejection.
- **Order:** a sale owed to a seller, with return-window end and status (delivered, refund requested, disputed, refunded).
- **Payout line:** one obligation in a run (usually one per seller), listing its orders. Release lines pay out reserve that is no longer at risk.
- **Decision:** Clear, Reduce, Hold or Block, with owed, reserve, sendable amount, route and the result of every check.
- **Transfer:** one attempt on the rail with a deterministic request ID and a status (processing, sent, paid, failed, unknown).
- **Policy:** thresholds the marketplace sets. **Record:** the append-only log of every observation, decision, approval and call.

Invariants (non-negotiable)

These are enforced in code and each one has a test. If one cannot be proven, nothing else matters.

- **No double payment.** Every attempt carries a request ID. A timeout is UNKNOWN, never "failed"; the executor looks the request ID up on Airwallex before anything else. Airwallex rejects a reused request ID within 7 days.
- **The model never moves money or edits identity.** Its output is an intent with quoted evidence. It can quarantine a payout; it cannot release one or change bank details.
- **Dry run first.** The full plan (who is paid, how much, what is kept back, held or blocked, and why) exists and is viewable before any money moves.
- **Totals tie out.** For every run: owed = sent + kept back + held + blocked, in each currency.
- **Reconciliation halts on drift.** The internal record is reconciled to Airwallex balances and transfer states every cycle; any mismatch stops execution and alerts rather than guessing. (From Ringfence.)
- **Money is integers.** Amounts are held in minor units (kobo, cents), never floating point.

The eight checks

#	Check	Fires when	Outcome
1	Duplicate guard	The same orders appear in another line, or were already sent	Block
2	Bank details	Airwallex validation rejects the beneficiary	Block
3	Bank response	The bank returned the last payment (e.g. name mismatch); stays until a person corrects it	Block
4	Bank-change quarantine	Details changed inside the cooling-off window, or an unverified change request arrived	Hold
5	Refund reserve	Orders are still returnable or disputed; keep back refunds + disputes + expected returns	Reduce (Hold if all at risk)
6	Approval limit	Sendable amount exceeds what Holdpoint may send alone	Hold until approved
7	Funding	Sending would take the wallet below its floor	Hold
8	Route	Always: picks local transfer or SWIFT, cheapest the details support	Informational

Precedence: Block over Hold over Reduce over Clear. Defaults: 14-day return window, 100% dispute reserve, 72h bank-change cooling-off, US\$10,000 approval limit, US\$25,000 wallet floor.

Agent behaviour

- **Observe** orders, refunds, beneficiaries, balances and existing transfers.
- **Read** seller emails into typed intents (bank-change request, dispute, payout query, other) with a confidence and a quoted sentence. If either the model or the rules reader sees a bank-change request, it is treated as one.
- **Decide** with the eight checks; produce the dry-run plan.
- **Execute** only Clear, Reduce and approved lines through Airwallex.
- **Reconcile** every transfer to a terminal state; handle failures and unknowns as above.
- **Re-decide** only the lines an event touches, and tell each affected seller in plain words.

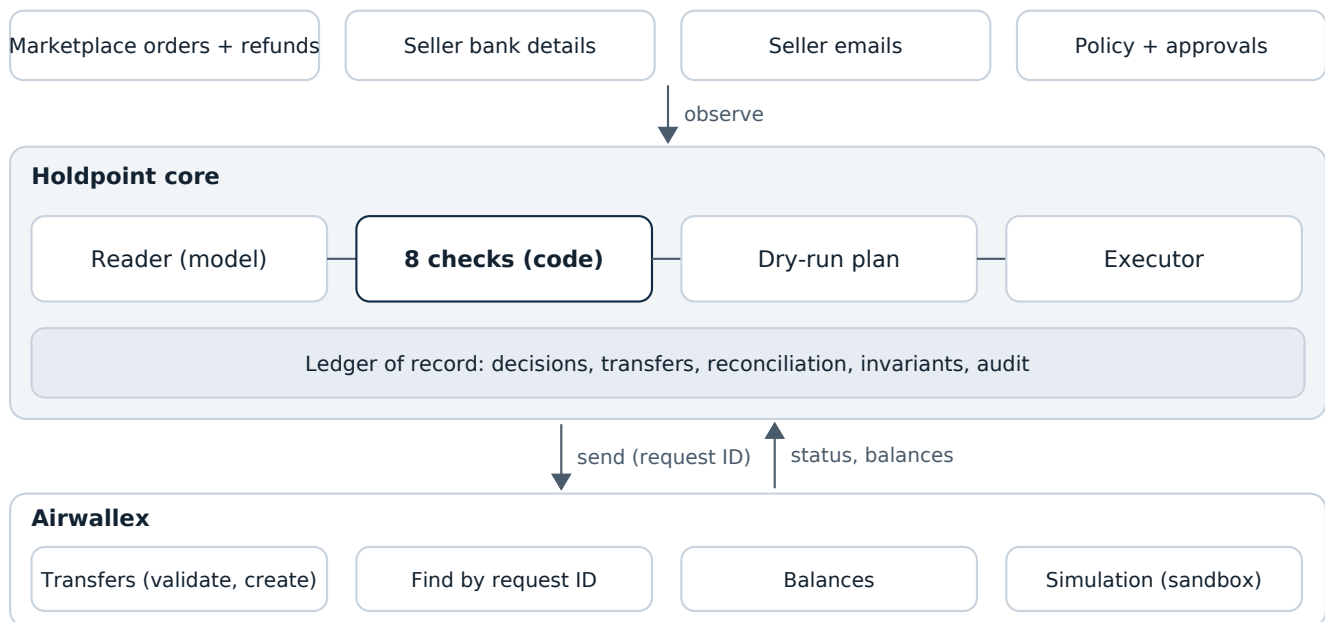
Airwallex integration

Step	Endpoint
Authenticate	POST /api/v1/authentication/login
Validate before sending	POST /api/v1/transfers/validate; beneficiary validation
Send a payout	POST /api/v1/transfers/create (request_id, beneficiary_id, amount, currencies, transfer_method)
Find after a timeout	GET /api/v1/transfers?request_id=...
Track status	GET /api/v1/transfers/{id}
Balances for reconciliation	GET /api/v1/balances/current
Sandbox events	POST /api/v1/simulation/transfers/{id}/transition (next_status, failure_type)
Tag incoming funds (later)	A Global Account per marketplace or funding source (from Ringfence)

Name-match verification covers SEPA, UK and Vietnam only; elsewhere Holdpoint relies on validation, history and quarantine.

Architecture

Inputs



Inputs are observed; the model only reads; eight deterministic checks produce a dry-run plan; only the executor talks to Airwallex, with a request ID on every call; the ledger of record wraps every step and halts on drift.

Operator console requirements

- A control-tower header: money that can move today against total owed, split into cleared, kept back, holding and blocked, with the send action and its dry-run totals.
- Four bays (Cleared, Reduced, Holding, Blocked) of payout strips, each showing owed, kept back, sends now, route and the reason inline. Strips move bays when a decision changes.
- A seller panel: every check and its inputs, orders and return windows, bank details, payment attempts, and every message sent to the seller.
- Approvals bound to the exact amount and account; bank-change confirmation; correct-and-revalidate.
- A proof view: totals tie out, zero duplicates, zero AI bank-detail edits, reconciliation status. (From Ringfence.)
- The decision record, filterable by seller, run and event.

Non-functional requirements

- **Correctness first:** invariants enforced in code and verified by tests.
- **Determinism:** same inputs, same plan.
- **Idempotency:** safe across retries and restarts.
- **Security:** Airwallex keys only on the server; API routes authenticated, rate-limited and validated.
- **Sandbox-safe:** nothing touches real money.
- **Accessible:** keyboard navigable, readable contrast, reduced motion respected.

Build phases and ownership

Phase	Scope	Owner
Done	Policy engine with 17 tests, simulated rail, operator console, landing page, PRD, flows, architecture	Anjola
1. Hardening	Integer money, server-side executor with Postgres ledger, request IDs stored before send, locked-down API routes	Anjola
2. Live sandbox	Airwallex keys, beneficiary setup, live transfers, simulated failures and timeouts	Anjola
3. Controls	Dry-run plan view, reconciliation that halts on drift, proof view	GRAMMYboy
4. Optimiser	Reserve sizing from refund history; cheapest route and FX timing within tolerance	GRAMMYboy
5. Demo	Naive-vs-Holdpoint comparison, video under 5 minutes, write-up	Both

Demo plan

- Payout day at a fictional marketplace: 14 payouts, 8 countries. The dry run lands most on Clear, some on Reduce, a few on Hold and Block, each with a reason.
- A buyer files a refund: one seller moves from Clear to Reduce; nothing else changes.
- An email asks to change a seller's bank account: the model reads it, the payout is held, the bank details are untouched.
- Send: a bank rejects one payment (blocked until corrected), and a response is lost (looked up by request ID, no second payment).
- A return window closes: the kept-back amount is released.
- The contrast beat (from Ringfence): the same inputs through a naive batch payout would have paid the duplicate row, the fraudster and the refundable money. Then the proof view: everything ties out.

Out of scope

- Real money beyond the Airwallex sandbox.
- KYC, tax, and seller onboarding.
- A seller-facing app (sellers receive messages only).
- Accounting and reconciliation after money lands (Leapfin's lane).